

## ranges::to<view>

- - Abstract
  - Revision history
  - Motivation
  - Discussion
  - Proposed change

### Abstract

This paper proposes to remove the constraint that `ranges::to` requires the target object not to be a `view`.

### Revision history

#### R0

Initial revision.

### Motivation

The requirement for `ranges::to` that the target object `C` should satisfy `input_range` was removed in [P1206R4](#), subsequently, LWG [3785](#) verified that such relaxation was intentional.

Among them, Barry gave an example of using `ranges::to<expected>` to transform `vector<expected>` into `expected<vector>`. Although currently `expected` does not have such a constructor, the example itself is indeed reasonable.

However, `ranges::to` still require that `C` cannot be a `view`, even though `C` now can be *any* class type. This constraint excludes very useful cases, chief among which is collecting split subranges into a `vector<string_view>`:

```
auto ints = "1.2.3.4"sv
| views::split('.')
| ranges::to<vector<string_view>>(); // error
```

Above, `ranges::to` recursively calls itself to construct `string_view` via `ranges::to<string_view>`, which is currently forbidden. However, this works:

```
auto ints = "1.2.3.4"sv
| views::split('.')
| ranges::to<vector>(); // ok
```

As CTAD deduces that the element type is `subrange`, there is really no difference between the two IMO. To achieve the first, we need a redundant lambda:

```
auto ints = "1.2.3.4"sv
| views::split('.')
| views::transform([](auto r) { return string_view(r); })
| ranges::to<vector>(); // ok
```

This is frustrating, it would be great if the first case could work *naturally*. Even more unfortunately, compared to the `string_view` case, the following code is well-formed in the current standard:

```
auto ints = "1.2.3.4"sv
| views::split('.')
| ranges::to<vector<span<const char>>>(); // ok in the current standard!!!
```

This is because the `span`'s range constructor is not `explicit`, so `subrange` can be implicitly converted to `span`, which in turn that `vector<span<const char>>` can be directly constructed from a range whose elements are `subranges` of `char` via range-tag constructor, without recursively going through `ranges::to`!

Such inconsistency is quite unsatisfactory, as splitting strings (whether via `split`, `slide`, `chunk`, `chunk_by`, etc.) into `string_views` is usually a common usage.

### Discussion

Now, the question becomes: why did we require that the target type cannot be a `view` in the first place? I believe this may be due to preventing users from making views this way or for safety concerns.

For the first case, it makes sense to me for those in `<ranges>` but not for `string_view/span`, given that `ranges::to<string_view>` is indeed an intuitive spelling. Even so, this constraint does not block `gsl::span`, `ranges::v3::moew_view`, or even `tuple<some-view-types>` since those are not model `ranges::view`.

I don't think any user would want to use `ranges::to` to construct `take_view` instead of just spell `views::take`, but even if they did, it wouldn't cause harm that warrants a special prohibition, just as the standard doesn't disallow the view's constructor for users.

If the purpose is to prevent dangling, it is indeed worth thinking about. Because in the current standard, `string_view` is allowed to be constructed from *rvalue* `string`, which means that if there is no such restriction, then `ranges::to<string_view>(string("abc"))` will be well-formed and definitely dangling.

But this is easy to resolve. We can require the input range to be `borrowed_range` when the target object is a view. That is to say, even if the input range is destroyed after the function call, its iterator is still valid, thus there is no dangling. Noted that this particular requirement should not be a function signature but a *Mandates*, since we do not know the type of the input range for the `to(Args&&)` overload at this point, which would lead this overload to be called accidentally when calling `ranges::to<string_view>(non-borrowed-range-types)`.

As a result, the requirements that the target type should not be `view` or the input type should be `borrowed_range` when the target type is `view` need be moved into *Mandates* of `to(R&&, Args&&...)`, which also resolves LWG [3985](#).

The following is the Tony table for the relaxation and I believes that this can be regarded as a DR for C++23.

| Before                                                                                                                                                                                                                                                                                                 | After P3544                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>auto sv1 = ranges::to&lt;string_view&gt;("abc"s );    // error auto sv2 = ranges::to&lt;string_view&gt;("abc"sv);    // error  array arr{...}; auto sp1 = ranges::to&lt;span&lt;int&gt;&gt;(std::move(arr)); // error auto sp2 = ranges::to&lt;span&lt;int&gt;&gt;(arr);           // error</pre> | <pre>auto sv1 = ranges::to&lt;string_view&gt;("abc"s );    // error auto sv2 = ranges::to&lt;string_view&gt;("abc"sv);    // ok  array arr{...}; auto sp1 = ranges::to&lt;span&lt;int&gt;&gt;(std::move(arr)); // error auto sp2 = ranges::to&lt;span&lt;int&gt;&gt;(arr);           // ok</pre> |
| <pre>auto take = views::iota(0)   ranges::to&lt;ranges::take_view&gt;(5);    // error  auto drop = vector&lt;int&gt;{...}   ranges::to&lt;ranges::drop_view&gt;(5);    // error</pre>                                                                                                                  | <pre>auto take = views::iota(0)   ranges::to&lt;ranges::take_view&gt;(5);    // ok, C1  auto drop = vector&lt;int&gt;{...}   ranges::to&lt;ranges::drop_view&gt;(5);    // error</pre>                                                                                                           |
| <pre>auto ints = "1.2.3.4"sv   views::split('.')   ranges::to&lt;vector&lt;string_view&gt;&gt;();    // error</pre>                                                                                                                                                                                    | <pre>auto ints = "1.2.3.4"sv   views::split('.')   ranges::to&lt;vector&lt;string_view&gt;&gt;();    // ok</pre>                                                                                                                                                                                 |
| <pre>int numbers[] = {...}; auto groups1 = numbers   views::slide(3)   ranges::to&lt;vector&lt;span&lt;int&gt;&gt;&gt;();    // ok  auto groups2 = numbers   views::slide(3)   ranges::to&lt;vector&lt;span&lt;int, 3&gt;&gt;&gt;(); // error</pre>                                                    | <pre>int numbers[] = {...}; auto groups1 = numbers   views::slide(3)   ranges::to&lt;vector&lt;span&lt;int&gt;&gt;&gt;();    // ok  auto groups2 = numbers   views::slide(3)   ranges::to&lt;vector&lt;span&lt;int, 3&gt;&gt;&gt;(); // ok, r</pre>                                              |
| <pre>auto views = vector&lt;int&gt;{...}   views::transform([](int x) { return to_string(x); })   ranges::to&lt;vector&lt;string_view&gt;&gt;();    // ok, but dangling</pre>                                                                                                                          | <pre>auto views = vector&lt;int&gt;{...}   views::transform([](int x) { return to_string(x); })   ranges::to&lt;vector&lt;string_view&gt;&gt;();    // ok, t</pre>                                                                                                                               |

## Proposed change

This wording is relative to [N5001](#).

1. Modify 25.2 [\[ranges.syn\]](#) as indicated:

```
#include <compare>           // see [compare.syn]
#include <initializer_list>   // see [initializer.list.syn]
#include <iterator>           // see [iterator.synopsis]

namespace std::ranges {
    [...]
    // [range.utility.conv], range conversions
    template<class C, input_range R, class... Args> requires (!view<C>)
        constexpr C to(R&& r, Args&&... args); // freestanding
    template<template<class...> class C, input_range R, class... Args>
        constexpr auto to(R&& r, Args&&... args); // freestanding
    template<class C, class... Args> requires (!view<C>)
        constexpr auto to(Args&&... args); // freestanding
    template<template<class...> class C, class... Args>
        constexpr auto to(Args&&... args); // freestanding
    [...]
}
```

2. Modify 25.5.7.2 [\[range.utility.conv.to\]](#) as indicated:

```
template<class C, input_range R, class... Args> requires (!view<C>)
constexpr C to(R&& r, Args&&... args);
```

-1- *Mandates*:

(2.1) — C is a cv-unqualified class type.

(?.?) — C does not satisfy `view` or R models `borrowed_range`.

3. Modify 25.5.7.3 [\[range.utility.conv.adaptors\]](#) as indicated:

```
template<class C, class... Args> requires (!view<C>)
constexpr auto to(Args&&... args);
template<template<class...> class C, class... Args>
constexpr auto to(Args&&... args);
```